Exercise 11: Reference Model and Scoreboard

Objective: *Introduction to reference models, scoreboards, TLM ports and exports.*

Task: *Implementing a reference model and scoreboard for the Saturation Filter.*

The files for this exercise are in `<ROOT>/lecture07/exercises/E11_sat_pyuvm_ref_model_scoreboard/`. This directory contains the RTL and testbench files. In the following sections, file names are relative to `tb/`.

To ensure correct behavior of the design, the reference model calculates the expected outcome, which is then compared against the actual outcome of the DUT in the scoreboard. In this exercise you will implement the reference model for the *Saturation Filter* and the checking logic in the scoreboard.

Both the reference model and the scoreboard use Transaction Level Modeling (TLM). They communicate using TLM ports. Figure 1 shows the *Saturation Filter* testbench and connections of the components with their respective port types. You can read more about TLM ports and export in the *PyUVM* compendium.

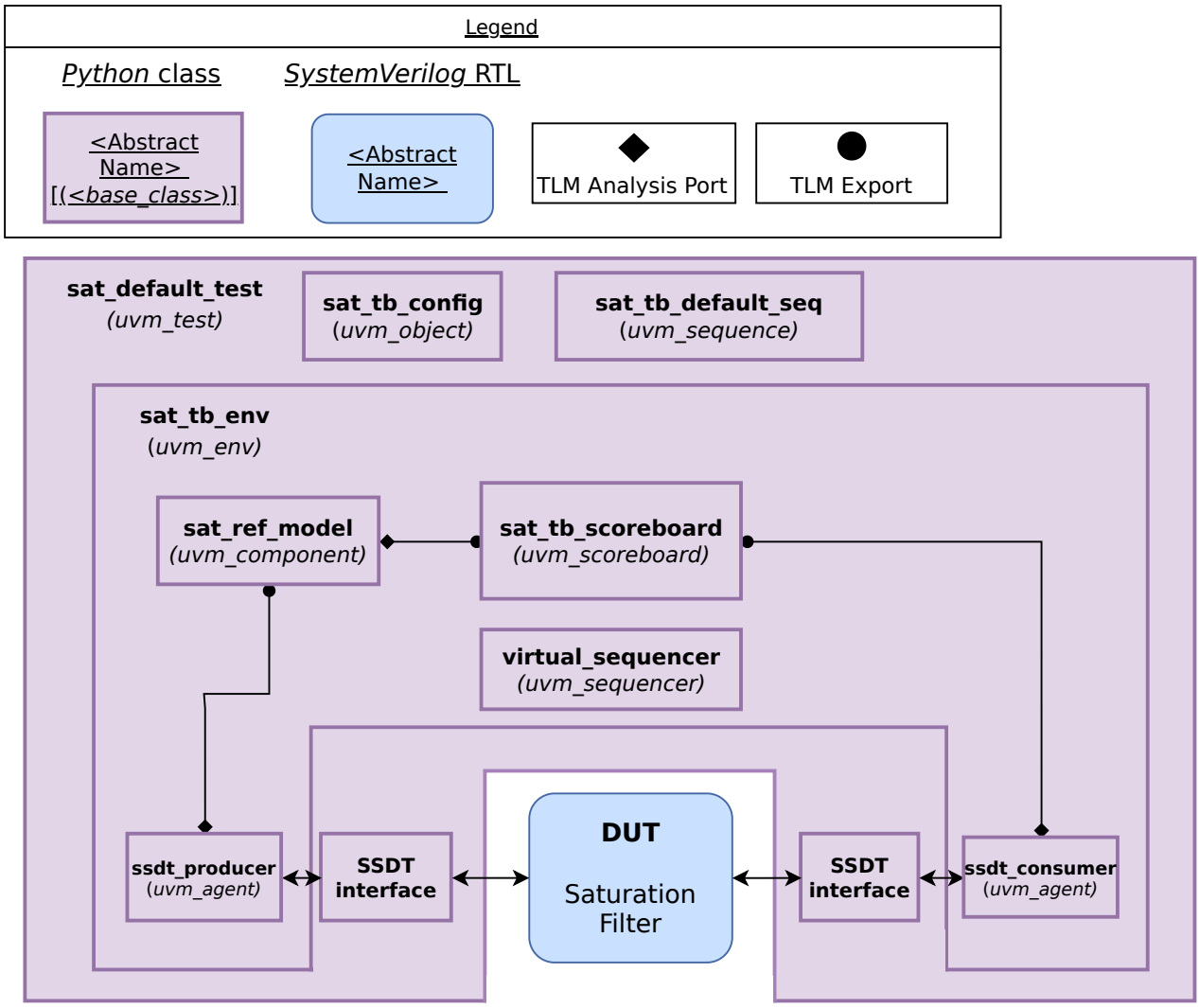


Fig. 1: Testbench setup after this exercise with added scoreboard and reference model.

Part 1: Reference Model

The reference model receives input transactions from the monitor of the producer uvc and produces the expected output transactions. It extends `uvm_subscriber`, which provides an analysis export that calls `write()` for each received transaction.

Part of the reference model is already implemented in `sat_ref_model`. In its current version the `write()` method only logs the received item. The reference model is already instantiated in the environment `build_phase`, and the producer agent's analysis port is connected to the reference model's analysis export in the `connect_phase`.

Run the tests and observe the transactions reported by the reference model. They are the transactions sampled by the producer monitor.

Complete `sat_ref_model.py` as follows:

- Create an `uvm_analysis_port` in the constructor.
- In `write()`
  - Clone the received item using the `clone()` method of the sequence item.
  - Calculate the expected output and set the `data` field in the cloned item.
  - Send the item through the analysis port using `write()`.

Part 2: Scoreboard

The scoreboard compares the expected transactions from the reference model with the transactions sampled at the DUT output. The skeleton in `sat_tb_scoreboard.py` extends `uvm_scoreboard` and provides two `uvm_tlm_analysis_fifo` instances: `ref_model_fifo` and `uvc_ssdt_consumer_fifo`. Read more about `tlm_analysis_fifo` in the *PyUVM* compendium.

Complete the scoreboard as follows: In `run_phase`, in a continuous (endless) loop""

- Get one transaction from each FIFO.
- Compare the transactions. Hint: `uvc_ssdt_seq_item` implements `__eq__` so the items can be compared with `==`.
- Increment `succes` for matching transactions. For mismatches, report an error and increment `failure`.
- In `check_phase`, report both counters and assert that `failure` is zero.

Integrate the scoreboard in `sat_tb_env.py`:

- Add a scoreboard handle and instantiate the scoreboard in `build_phase`.
- Connect the consumer agent's analysis port to `uvc_ssdt_consumer_fifo.analysis_export`.
- Connect the reference model's analysis port to `ref_model_fifo.analysis_export`.

After the scoreboard is integrated into the testbench, check if both the `success` and `failure` variables correctly count up for matching and mismatching items respectively. Consider also how to test if the scoreboard catches mismatched items. For example, inject transactions with errors into the scoreboard to see if it catches them.